

Motion planning with doors makes games hard

Manuel Frohn

RWTH Aachen University, Aachen, Germany
`manuel.frohn@rwth-aachen.de`

Abstract.

Keywords: PSPACE · Doors · Motion Planning · Games

1 Introduction

Many games, such as the Jump'n Runs of the Super Mario series, require the player to move from one point to another to complete a level. This kind of problem is referred to as 1-Player-Motion-Planning. In simple environments, where nothing but paths exist, this problem is easy, as it is equivalent to the well known s-t-Path Problem. But for most games it is not. One of the reasons for this, is the occurrence of doors devices in the games environment. A door device is a gadget, that consists out of three traversable tunnels and an inner state. One of the tunnels opens the door, one closes it and one acts as the "traverse" tunnel. This tunnel may only be traveled if and only if the door is opened. If such a device is part of the environment of a game we know the game to be PSPACE-hard i.e. An algorithm calculating a path as required by a game will do so by using a minimum of Polynomial-Space in relation to the input length.

2 Fundamentals

2.1 The Problem

The problem of concern here is 1-player-motion-planning (1). It is central to understand that 1-player-motion planning on games does not only look at real levels of the game but rather on any level possible in the games implied environment. This means any level is considered, that may be constructed out of the objects making up the game, like platforms, enemies, or general game mechanics such as gravity changes. Comparable, to how the s-t-path problem on general graphs does not only look at certain graphs but rather at all possible general graphs.

Definition 1 (1-Player-Motion-Planning).

1-Player-Motion-Planning is the problem of finding a path from a given point A to a given point B in a certain environments (such as a graph or a grid) that can be traversed by the games player.

2.2 The Concern

The main concern of this article is PSPACE-hardness. Just as with time complexity, which concerns itself with the runtime of an algorithm, space complexity concerns itself with the space used by an algorithm. And just as with time complexity, space complexity is divided into different complexity classes. The main one of concern here is the class of PSPACE-hard (2) problems. The problems that require **more** than polynomial space to be solved.

Definition 2 (PSPACE-hardness).

*A problem is PSPACE-hard if and only if, the amount of space a optimal algorithm requires to solve the problem is at **least** polynomial in relation to the input length.*

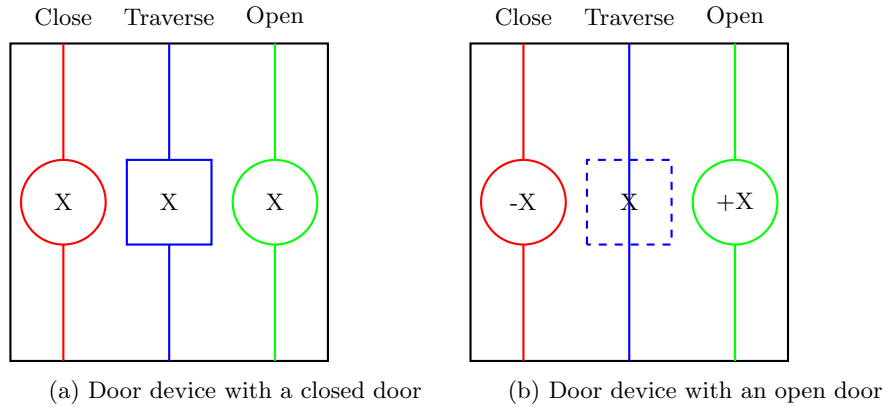
And just as NP has SAT as its canonical problem PSPACE also has a canonical problem, the quantified version of SAT (3) to be precise. There, not only the usage of the logical-or and logical-and is allowed, but also the usage of the quantifiers exists ($\exists$) and forall ($\forall$).

Definition 3 (Quantified-SAT).

Quantified-SAT is the canonical problem of PSPACE. It is an extension of the original SAT problem, expanding the problem by additionally allowing quantified-boolean-algebra expressions. For example $\exists x \forall y : x \wedge y \wedge z$

3 The door device

A door device is a construct made out of three main components: A traverse tunnel, that may only be traversed if the door within it is open, an open tunnel, with a pressure plate which opens the door on being traversed and a close tunnel with a pressure plate which closes the door on being traversed.



4 PSPACE-hardness of doors

To prove PSPACE-hardness for door devices, we will prove a more general theorem(1) concerning doors and their effect on the space complexity of 1-player-motion-planning. This theorem is proven by reducing from True-Quantified-SAT to 1-Player-Motion-Planning with doors. We will then use the theorem to prove PSPACE-hardness for the generic door device, and all the variants, that will be introduced later.

Theorem 1 (Doors are PSPACE-hard).

1-Player-Motion-Planning is PSPACE-hard in an environment that contains doors, that each are controlled by an open and a close pressure plate.

4.1 The reduction

The reduction works by simulating the components of a True-Quantified-SAT problem, and then assembling the complete simulation out of these components. In the simulation the player has to first pass the quantifier devices upper part, satisfying their condition on the equation, by setting the truth value of the associated variable. After that the player needs to traverse the clause gadgets, which are only traversable if one of the associated variables is set accordingly to the clause in the original problem. After the player has traversed the clause gadgets, he needs to traverse the lower part of the quantifier gadgets. This serves as a way for the All-Quantifier to test both variable settings, by forcing the player to go through the upper part again (from the current gadget onward, only) with the associated variable being set the other opposite value. Figure 2 shows such a higher level view of the simulation of this True-Quantified-SAT equation:

$$\forall x \exists y \exists z : (\bar{x} \vee y \vee z) \wedge (\bar{x} \vee \bar{y} \vee z) \wedge (x \vee y \vee \bar{z})$$

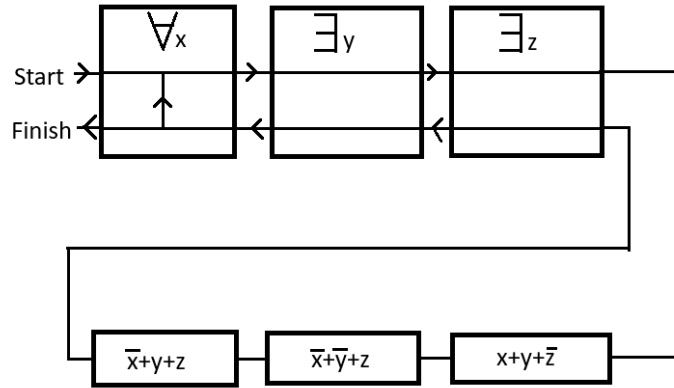
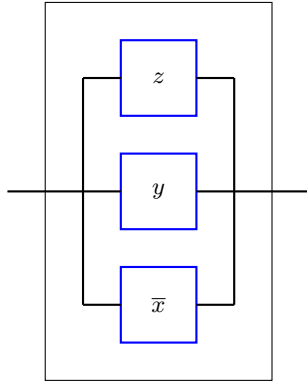


Fig. 2: Simulation of a True-Quantified-SAT equation through 1-Player-Motion-Planing with doors

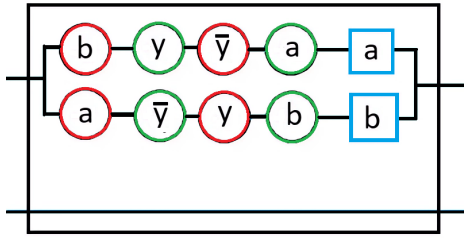
4.2 The clause gadget

The clause gadget is made up of doors, representing the truth values required by the clause to be true. Therefore, if in the quantifier part of the simulation a variable is set as required by the clause, at least one of the doors is open, enabling the player to advance. Creating the behavior we require, as specified in the prior subsection. Figure 3 shows such a clause gadget simulating the clause: $(\bar{x} \vee y \vee z)$

Fig. 3: A clause gadget simulating the clause $(\bar{x} \vee y \vee z)$

4.3 The Exists-Quantifier gadget

The Exists-Quantifier gadget consists out of a upper and a lower part. The upper part itself also consists out of two paths the first one opens the doors associated with one truth value variable of the associated variable, while the other one opens the doors with the inverse truth value. Also both paths close the doors associated with the opposite truth value of the variable. This creates the behavior of letting the player choose the truth value of the variable, as required by the Exists-Quantifier. Now the lower part serves only as a pass through path, for the agent to get to the finish line or into the next ForAll-Quantifier.

Fig. 4: A Exists-gadget simulation $\exists y$

4.4 The forAll-Quantifier gadget

The forAll-Quantifier gadget consists out of three parts, the upper, the lower and the middle part. The upper when traversed locks, the lower part and sets the variable associated with the gadget to true (by opening the associated doors).

The lower part serves the same purpose as in the Exists-Quantifier just that contains a door and is therefore only traversable if that door is open. The middle part can only be traversed from the start of the lower part, to the end of the upper part. It inverts the variables truth value and opens the lower part. This assembly ensures that both variable truth values have to be tested, before the player can proceed onwards to the finish line.

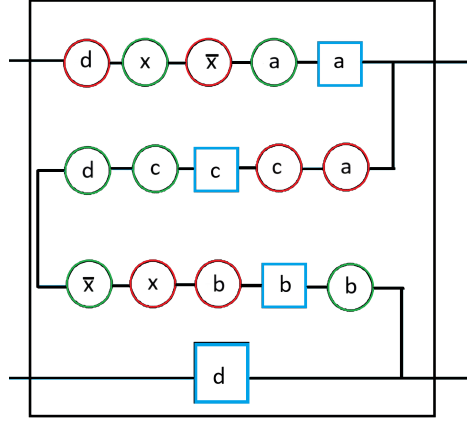


Fig. 5: A Exists-gadget simulation $\forall x$

5 Variants of door devices